

UTU - DGETP
Tecnólogo en Ciberseguridad
Instituto Tecnológico de Informática

Documentación del Proyecto Final

Sistema de registro e ingreso seguro de usuarios

Asignatura	Introducción a la programación
Proyecto	Programa de autenticación con validación, hashing, salt y pregunta de seguridad
Integrante/s	Pablo Dominguez
Profesor	Andrés Álvarez
Grupo	TM1
Fecha de entrega	23/06/2026

Índice

Índice	2
1. Introducción	3
2. Objetivo del programa	3
3. Soluciones que ofrece en ciberseguridad	3
4. Estructura modular del proyecto	4
5. Incorporación de conceptos del curso	4
6. Conclusión	5
7. Código fuente del programa	5
menues.py	5
validaciones.py	8
seguridad.py	14

1. Introducción

El presente documento describe el proyecto final desarrollado para la asignatura Introducción a la programación. El programa implementa un sistema de registro e ingreso de usuarios orientado al área de la ciberseguridad. Su objetivo principal es aplicar conceptos básicos de programación imperativa en un caso práctico relacionado con la protección de credenciales.

El sistema permite registrar usuarios, validar requisitos mínimos de seguridad, transformar contraseñas mediante un algoritmo de hashing casero con salt, guardar datos en un archivo de autenticación y verificar el ingreso posterior del usuario.

2. Objetivo del programa

El objetivo del programa es simular un sistema simple de autenticación segura. Para ello se evita almacenar contraseñas en texto claro y se utilizan mecanismos básicos de validación de usuario, contraseña y preguntas de seguridad.

El proyecto fue desarrollado en Python, un lenguaje imperativo que permite trabajar con variables, estructuras de control, funciones, listas, archivos y modularización.

3. Soluciones que ofrece en ciberseguridad

El programa ofrece soluciones educativas vinculadas a la protección de cuentas de usuario. No pretende reemplazar un sistema profesional de autenticación, pero sí demuestra principios fundamentales de seguridad.

- Gestión de usuarios: permite registrar usuarios y luego validar su ingreso.
- Validación de políticas de usuario: exige longitud mínima, uso de mayúsculas y números.
- Validación de contraseñas: controla longitud, coincidencia de confirmación, mayúsculas y números.
- Hashing de contraseñas: la contraseña no se guarda directamente, sino convertida a un valor hexadecimal.
- Uso de salt: cada contraseña se combina con una cadena aleatoria para evitar que dos contraseñas iguales produzcan necesariamente el mismo resultado.
- Persistencia en archivo: los usuarios registrados se guardan en auth.txt para poder ser cargados en futuras ejecuciones.
- Pregunta de seguridad tipo captcha educativo: antes de registrar o ingresar, el usuario debe responder una pregunta vinculada a conceptos básicos de ciberseguridad.
- Manejo de errores: se informa al usuario cuando los datos ingresados no cumplen las condiciones esperadas.

4. Estructura modular del proyecto

El programa se divide en módulos para separar responsabilidades. Esto mejora la organización del código y permite que cada archivo cumpla una función específica.

Archivo	Responsabilidad
menues.py	Contiene el menú principal, la interacción por consola, la selección de opciones y la llamada a las funciones de validación.
validaciones.py	Contiene las listas de usuarios, hashes y salts; carga y guarda datos en archivo; valida registro y login; realiza la búsqueda recursiva de usuarios; selecciona preguntas de seguridad.
seguridad.py	Contiene las funciones relacionadas con seguridad: generación de salt y cálculo del hash casero de 16 bytes.
auth.txt	Archivo de datos utilizado para persistir usuarios registrados con el formato usuario hash salt. No es código fuente.

5. Incorporación de conceptos del curso

La consigna solicita que el programa incorpore al menos el 50% de los conceptos propuestos en los temas del curso. El proyecto aplica contenidos de la mayoría de las unidades.

Tema del curso	Concepto aplicado	Ejemplo en el programa
1. Introducción a la computación	Algoritmo, sintaxis, semántica, ejecución.	El sistema sigue pasos definidos para registrar, validar, guardar y autenticar usuarios.
2. Programación imperativa	Variables, constantes, tipos de datos, asignación, entrada y salida.	Uso de variables usuario, contraseña, respuesta; input y print para interactuar con el usuario.
3. Estructuras de control	Secuencia, selección, iteración y recursión.	while True, if/else, match/case, for y función recursiva buscar usuario.
4. Estructuras de datos	Tipos primitivos y estructurados.	Listas, diccionarios, cadenas, números enteros y archivo de texto.
5. Modularización	Funciones, parámetros y separación por módulos.	menues.py, validaciones.py y seguridad.py separan interfaz, validación y seguridad.

6. Búsqueda	Búsqueda lineal.	buscar_usuario recorre la lista de usuarios hasta encontrar coincidencia o finalizar.
7. Calidad y corrección	Manejo de errores y buenas prácticas básicas.	try/except para entradas numéricas, mensajes de error acumulados y validaciones antes de registrar.

En total se incorporan contenidos de los 7 temas del curso, por lo que se supera el mínimo solicitado del 50%.

6. Conclusión

El proyecto permite aplicar contenidos centrales de programación imperativa en un problema vinculado a la ciberseguridad. A través del registro, validación, hashing, uso de salt, búsqueda recursiva y almacenamiento en archivo, el programa demuestra cómo se puede construir un sistema básico de autenticación.

Además, la estructura modular facilita la comprensión del código y separa las responsabilidades principales del sistema: interacción con el usuario, validaciones y funciones de seguridad.

7. Código fuente del programa

A continuación se incorpora el código fuente del programa, separado por archivo.

menues.py

```
from validaciones import *

def menu_ingreso():
    while True:

print("-----")
print("1. Registrarse")
print("2. Loguarse")
print("3. Salir")
```

```

print("-----")
print("-----")

        try:
            opcion = int(input("                                Ingtrese
una opcion: "))

print("-----")
print("-----")

        match opcion :

            case 1 :
                usuario = input("
Ingtrese nombre de usuario: ")

print("-----")
print("-----")
                contraseña = input("
Ingtrese contraseña: ")

print("-----")
print("-----")
                confirm_contraseña = input("
Ingtrese nuevamente la contraseña: ")

print("-----")
print("-----")
                pregunta = elegir_pregunta()
                respuesta = menu_pregunta(pregunta)

print("-----")
print("-----")
                registro = validaciones_registro(
                    usuario,
                    contraseña,
                    confirm_contraseña,
                    pregunta,
                    respuesta
                )
                if registro != True:
                    for mensaje in registro:
                        print(mensaje)
                    menu_ingreso()
                else:

```

```

                                print(f"                Registro realizado con
exito, bienvenido al club, {usuario}")

        case 2 :
            usuario = input("
Ingrese nombre de usuario: ")

print("-----")
            contraseña = input("
Ingrese contraseña: ")

print("-----")

            pregunta = elegir_pregunta()
            respuesta = menu_pregunta(pregunta)
            logueo = validaciones_login(
                usuario,
                contraseña,
                pregunta,
                respuesta
            )
            if logueo != True:
                for mensaje in registro:
                    print(mensaje)
                menu_ingreso()
            else:
                print(f"                Login realizado con
exito, bienvenido al club, {usuario}")

        case 3 :
            return

        case _ :

print("-----")
            print("Opcion invalida")

print("-----")

except ValueError:
    print("El valor ingresado debe ser un número")

```

```

def menu_pregunta(pregunta):

    while True:

        print("                ", pregunta["pregunta"])

print("-----")
print("-----")
        print(pregunta["opcion 1"])
        print(pregunta["opcion 2"])
        print(pregunta["opcion 3"])
        print(pregunta["opcion 4"])

print("-----")
print("-----")

        try:

                respuesta = str(input("                Ingrese
una respuesta: "))

                if respuesta not in ["A", "B", "C", "D"]:

print("-----")
print("-----")
                        print("                Respuesta
invalida")

print("-----")
print("-----")
                                menu_pregunta(pregunta)

                                return respuesta

        except ValueError:
                print("Se debe ingresar una letra mayuscula de la A
a la D")

menu_ingreso()

#python -m venv venv
#venv\Scripts\activate
#python -m pip install pyinstaller
#pyinstaller --onefile menues.py

```

validaciones.py

```

from seguridad import *

```



```

# Lista de usuarios registrados.
usuarios = []

# Lista con las contraseñas hasheadas de los usuarios
registrados.
contraseñas_hasheadas = []

# Lista de salts para los hasheos.
salts = []

# Acumulador de mensajes de error para retornar al usuario.
mensaje_error = []

# Carga los usuarios guardados en auth.txt a las listas en
memoria.
def cargar_usuarios_desde_archivo():
    try:
        # Abre el archivo "auth.txt".
        with open("auth.txt", "r", encoding="utf-8") as archivo:
            # Lee cada línea del archivo.
            for linea in archivo:
                # Crea un array "datos" y guarda los datos del
                usuario separados por una barra recta.
                datos = linea.strip().split("|")

                # Guada los datos del usuario en variables
                separadas.

                usuario = datos[0]
                contraseña_hasheada = datos[1]
                salt = datos[2]

                # Agrega sus datos a las variables locales.
                usuarios.append(usuario)

                contraseñas_hasheadas.append(contraseña_hasheada)
                salts.append(salt)

    # Si no encuentra el archivo creado, lo crea y lo cierra.
    except FileNotFoundError:
        archivo = open("auth.txt", "w", encoding="utf-8")
        archivo.close()

# Guarda un usuario nuevo en auth.txt.
def guardar_usuario_en_archivo(usuario, contraseña_hasheada,
salt):

```

```

with open("auth.txt", "a", encoding="utf-8") as archivo:

archivo.write(f"{usuario}||{contraseña_hasheada}||{salt}\n")

# Funcion que sirve para validar las credenciales ingresadas
por el usuario para su registro.
def validaciones_registro(
    usuario,
    contraseña,
    confirm_contraseña,
    pregunta,
    respuesta
):

    # Valida que el nombre de usuario no se encuentre ya
registrado.
    if usuario in usuarios:
        mensaje_error.append("Este nombnre de usuario ya esta
registrado")

    # Valida que el nombre de usuario tenga entre 6 y 50
caracteres.
    if len(usuario) < 6 or len(usuario) > 50:
        mensaje_error.append("          El nombre de usuario debe
tener entre 6 caracteres y 50")

    # Valida que el nomnre del usuario tenga al menos una
letra mayuscula.
    if not any(letra.isupper() for letra in usuario):
        mensaje_error.append("El nomnre de usuairo debe contener
al menos una letra mayuscula")

    # Valida que el nombre del ususario tenga al menos un
numero.
    if not any(letra.isdigit() for letra in usuario):
        mensaje_error.append("El nombre de usuario debe contener
al menos un número")

    # Valida que la contraseña tenga entre 6 y 50 caracteres.
    if len(contraseña) < 6 or len(contraseña) > 50:
        mensaje_error.append("La contraseña debe tener entre 6
caracteres y 50")

    # Valida que las contraseñas coincidan.
    if contraseña != confirm_contraseña:
        mensaje_error.append("Las contraseñas no coinciden")

```

```

# Valida que la contraseña tenga al menos una letra
mayuscula.
    if not any(letra.isupper() for letra in contraseña):
        mensaje_error.append("La contraseña debe contener al
menos una letra mayuscula")

# Valida que la contraseña tenga al menos un numero.
    if not any(letra.isdigit() for letra in contraseña):
        mensaje_error.append("La contraseña debe contener al
menos un número")

# Valida que la respuesta a la pregunta de ingreso sea
correcta.
    if respuesta != pregunta["respuesta_correcta"]:
        mensaje_error.append("Respuesta ingresada incorrecta")

# Evalua si existen mensajes de error. Si existen, los
retorna y limpia el registro de errores.
    if mensaje_error:
        errores = mensaje_error.copy()
        mensaje_error.clear()
        return errores

# Si no hay errores, registra el usuario. Guarda su
contraseña hashada con salt, el salt lo guarda para poder
validar ingresos futuros.
    else:
        usuarios.append(usuario)
        contraseña_hashada = hash_casero_16_bytes(contraseña)
        contraseñas_hashadas.append(contraseña_hashada[0])
        salts.append(contraseña_hashada[1])
        guardar_usuario_en_archivo(usuario,
contraseña_hashada[0], contraseña_hashada[1])
        return True

# Funcion para validar ingreso.
def validaciones_login(
    usuario,
    contraseña,
    pregunta,
    respuesta
):
    # Evalua que la respuesta del captcha sea correcta
    if respuesta == pregunta["respuesta_correcta"]:
    # Busca el usuario en la lista de usuarios.
        posicion = buscar_usuario(usuarios, usuario, 0)
        # Evalua si el usuario ingresado existe.
        if posicion != -1:

```

```

        # Si existe, trae su contraseña hasheada
        utilizando la posicion del nombre de usuario en la lista de
        usuarios(porque la posicion del nombre de usuario y de su
        contraseña son las mismas en ambas listas).
        contraseña_hasheada_real =
        contraseñas_hasheadas[posicion]
        # Tambien traer el salt que uso para hashear la
        contraseña usando la misma logica.
        salt = salts[posicion]
        # Prepara la contraseña ingresada por el usuario
        para compararla con la contraseña guardada en el registro.
        contraseña_hasheada =
        hash_casero_16_bytes_login(contraseña, salt)
        # Evalua que ambas contraseñas sean iguales.
        if contraseña_hasheada[0] ==
        contraseña_hasheada_real:
            # Si lo son, la funcion retorna "True".
            return True
            # Si no son iguales carga mensaje de error en la
            lista de errores.
        else:
            mensaje_error.append("La contraseña ingresada no
            coincide, pruebe nuevamente")
            # Si el nombre de usuario no coincide lo carga en la
            lista de errores.
        else:
            mensaje_error.append("El nombre de usuario es
            incorrecto")
            # Si la respuesta es incorrecta la carga en la lista de
            errores.
        else:
            mensaje_error.append("Respuesta ingresada incorrecta")

        # Retorna la lista de errores.
        errores = mensaje_error.copy()
        mensaje_error.clear()
        return errores

# Funcion recursiva para buscar usuarios, esta funcion recibe
una lista, un usuario y una posicion.
def buscar_usuario(
    lista,
    usuario_buscado,
    posicion
):

```

```

# Si la posicion brindada es mayor al numero maximo de
posiciones en la lista, le resta 1.
if posicion >= len(lista):
    return -1

# Si el elemento que se encuentra, dentro de la lista, en
la posicion brindada es igual el nombre del usuario brindado,
retorna la posicion en la lista de ese usuario.
if lista[posicion] == usuario_buscado:
    return posicion

# Si ninguna de las dos cosas pasa, vuelve a ejecutrase
sumando 1 a la ultima posicion evaluada.
return buscar_usuario(
    lista,
    usuario_buscado,
    posicion + 1
)

```

```
def elegir_pregunta():
```

```

    preguntas = [
        {
            "pregunta": "¿Qué es un firewall?",
            "opcion 1": "A) Un programa para editar imágenes",
            "opcion 2": "B) Una barrera que filtra tráfico de
red",
            "opcion 3": "C) Un virus informático",
            "opcion 4": "D) Un tipo de memoria RAM",
            "respuesta_correcta": "B"
        },
        {
            "pregunta": "¿Para qué sirve el hashing de
contraseñas?",
            "opcion 1": "A) Para guardar la contraseña en texto
claro",
            "opcion 2": "B) Para convertir la contraseña en un
valor no reversible",
            "opcion 3": "C) Para hacer la contraseña más corta",
            "opcion 4": "D) Para enviarla por email",
            "respuesta_correcta": "B"
        },
        {
            "pregunta": "¿Qué es un salt en hashing de
contraseñas?",

```

```

        "opcion 1": "A) Un dato aleatorio que se suma antes
de hashear",
        "opcion 2": "B) Una contraseña temporal",
        "opcion 3": "C) Un tipo de virus",
        "opcion 4": "D) Una copia de seguridad",
        "respuesta_correcta": "A"
    },

    {
        "pregunta": "        ¿Qué significa phishing?",
        "opcion 1": "A) Un ataque para engañar al usuario y
robar información",
        "opcion 2": "B) Un tipo de antivirus",
        "opcion 3": "C) Una forma de comprimir archivos",
        "opcion 4": "D) Un protocolo de red",
        "respuesta_correcta": "A"
    }
]

pregunta_actual = random.choice(preguntas)

return pregunta_actual

# Cuadno se carga servicios.py, se carga la lista de usuarios
del archivo "auth.txt".
cargar_usuarios_desde_archivo()

```

seguridad.py

```

# Librerias importadas
import random

# Esta funcion sirve para generar un cadena de 16 caracteres
al azar
def salt_casero(largo=16):
    caracteres =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    salt = ""

    # Realiza 16 iteraciones sobre la lista de caracteres y en
cada una guarda uno al azar.
    for i in range(largo):
        posicion = random.randint(0, len(caracteres) - 1)
        salt += caracteres[posicion]

    return salt

```

```

# Esta funcion sirve para convertir la contraseña que ingresa
el usuario en una cadena de caracteres unica.
def hash_casero_16_bytes(contraseña):

    # Crea un salt especifico para cada hasheo
    salt = salt_casero()

    # Crea una lista de 16 números usando el salt
    hash_bytes = []

    # Convierte cada caracter del salta a su numero
    correspondiente en la tabla ascii
    for caracter in salt:
        hash_bytes.append(ord(caracter) % 256)

    # Itera sobre cada caracter y su indeice en la contrasena
    ingresada por el usuario.
    for i, caracter in enumerate(contraseña):

        # Convierte el caracter a su numero correspodniete en
        la tabla ascii
        numero = ord(caracter)

        # Itera 16 veces, generando un caracter aleatorio
        utilizando el salt para cada caracter de la contraseña, devuelve
        siempre 16 caracteres independientemente del largo de la
        contraseña ingresada.
        for j in range(16):
            numero_salt = ord(salt[j])
            hash_bytes[j] = (
                hash_bytes[j] * 71
                + numero * (j+1)
                + numero_salt * (j+1)
                + i * 17
                + j * 13
            )%256

    # Retorna la contrasena hasheada en formato haxadecimal de
    16 bytes y el salt que se uso para hashear la contraseña.
    return bytes(hash_bytes).hex(), salt

# Esta funcion es practiamente igual que la otra solo que en
vez de crear el salt utiliza el que el usario utilizo para
registrarse.
def hash_casero_16_bytes_login(contraseña, salt):

    hash_bytes = []

```

```

for caracter in salt:
    hash_bytes.append(ord(caracter) % 256)

for i, caracter in enumerate(contraseña):
    numero = ord(caracter)

    for j in range(16):
        numero_salt = ord(salt[j])
        hash_bytes[j] = (
            hash_bytes[j] * 71
            + numero * (j+1)
            + numero_salt * (j+1)
            + i * 17
            + j * 13
        ) % 256
    return bytes(hash_bytes).hex(), salt

```